

CSCI 5523 Introduction to Data Mining

Spring 2026

Assignment 4 (10 points)

Deadline for Part 1: April 20th 11:59 PM CDT

Deadline for Part 2: April 29th 11:59 PM CDT

**If you submit both Part 1 and Part 2 by April 20th (by 11:59 PM CDT),
you will receive 50% bonus points for your HW4 score**

1. Overview of the Assignment

In Assignment 4, you will create a content-based and item-based collaborative filtering recommendation system.

2. Requirements

2.1 Programming Requirements

- You must use **Python** to implement all tasks. You are allowed to use standard **Python libraries** and **numpy** (i.e., other external libraries like pandas are not allowed).
- You are required **to only use Spark RDD**, i.e., no point if using Spark DataFrame or DataSet.

2.2 Submission Platform

We will use Gradescope to run and grade your submission automatically. We highly recommend that you first test your scripts on your local machine before submitting your solutions.

2.3 Programming Environment

Python 3.9.12 and Spark 3.2.1

2.4 Write your own code

Do not use large language models (e.g., ChatGPT, Copilot) or share code with other students!

For this assignment to be an effective learning experience, **you must write your own code**. We emphasize this point because you will be able to find Python implementations of some of the required functions on the web. Please do not look for or at any such code.

TAs will combine all the code we can find from the web (e.g., GitHub), other students' code from this and other (previous) sections, and code generated from AI tools for plagiarism detection. **We will report all detected plagiarism to the university.**

3. Yelp Data

For this assignment, we generated a subset of the original Yelp review dataset using filters, such as "state" == "CA". We randomly split the sampled reviews into 80% for training, 10% for validation, and 10% for the test dataset. We do not share the test dataset. You can access and download the data on Google Drive:

- **train_review.json**

- **train_review_text_150k.json**
- **val_review.json** - containing only the target user and business pairs for prediction tasks

Note that you will use **train_review_text_150k.json** and **val_review.json** for **task 1**, and **train_review.json** and **val_review.json** for **task 2**.

4. Content-based Collaborative Filtering Recommendation System (5pts)

You need to submit the **2 Python scripts** on Gradescope (HW4 part 1):

- **task1_build.py, task1_predict.py** (all in lowercase)

Please do not change the content shown in the screenshots (Figure 1 and Figure 2) in your submission beyond adding your code to the main functions. You need to implement the main functions.

```
import pyspark
import argparse
import json
import math
import time
import string
from collections import Counter

def main(train_file, model_file, stopwords_file, sc):
    """
    Your code is here
    """

if __name__ == '__main__':
    start_time = time.time()

    sc_conf = pyspark.SparkConf() \
        .setAppName('hw4_task1') \
        .setMaster('local[*]') \
        .set('spark.driver.memory', '4g') \
        .set('spark.executor.memory', '4g')
    sc = pyspark.SparkContext(conf=sc_conf)
    sc.setLogLevel('OFF')

    parser = argparse.ArgumentParser(description='hw4-task2-build')
    parser.add_argument('--train_file', type=str, default='train_review_text_150k.json')
    parser.add_argument('--model_file', type=str, default='task1.model')
    parser.add_argument('--stopwords_file', type=str, default='stopwords')
    parser.add_argument('--time_file', type=str, default='task1_build.time')
    args = parser.parse_args()

    main(args.train_file, args.model_file, args.stopwords_file, sc)
    sc.stop()

    with open(args.time_file, 'w') as f:
        json.dump({'time': time.time() - start_time}, f)
    print('Duration:', time.time() - start_time)
```

Figure 1: task1_build.py template

```

import pyspark
import argparse
import json
import math
import time

def main(test_file, model_file, output_file, sc):
    """
    Your code is here
    """

if __name__ == '__main__':
    start_time = time.time()

    sc_conf = pyspark.SparkConf() \
        .setAppName('hw4_task1') \
        .setMaster('local[*]') \
        .set('spark.driver.memory', '4g') \
        .set('spark.executor.memory', '4g')
    sc = pyspark.SparkContext(conf=sc_conf)
    sc.setLogLevel('OFF')

    parser = argparse.ArgumentParser(description='hw4-task1-predict')
    parser.add_argument('--test_file', type=str, default='val_review.json')
    parser.add_argument('--model_file', type=str, default='task1.model')
    parser.add_argument('--output_file', type=str, default='task1.val.out')
    parser.add_argument('--time_file', type=str, default='task1_predict.time')
    args = parser.parse_args()

    main(args.test_file, args.model_file, args.output_file, sc)
    sc.stop()

    with open(args.time_file, 'w') as f:
        json.dump({'time': time.time() - start_time}, f)
    print('Duration:', time.time() - start_time)

```

Figure 2: task1_predict.py template

4.1. Task description

In this task, you will build a content-based recommendation system by generating profiles from review texts for users and businesses in the *train_review_text_150k.json*. Then you will use the model to predict if a user prefers to review a given business in *val_review.json* by computing the Jaccard similarity between the user and business profile vectors.

During the training process, you will construct the business and user profiles as follows:

- Step 1: Construct business and user profiles from review text (task1_build.py)
- Step 2: Predict user-business preferences using Jaccard similarity (task1_predict.py)

Step 1: Construct business and user profiles

Instructions for task1_build.py: You will write functions to build TF-IDF-based profiles for each business and user, and save them to **task1.model**.

- 1) Concatenate all reviews for a business into a single document, then parse it by removing punctuation, numbers, and stopwords. Remove extremely rare words whose frequency is less than **0.0001** of the total number of words.
 - a) Stopwords are provided
- 2) Measure word importance using TF-IDF, i.e., term frequency multiply inverse doc frequency
- 3) Use the **top 100 words** with the highest TF-IDF scores to represent each business document
- 4) Create a word set using these top 100 words as the business profile
- 5) Create a word set for the user profile by taking the **union** of all business profiles for businesses the user has reviewed
 - a) Unlike business profiles, which are fixed at 100 words, user profiles have no size limit. A user who reviewed many businesses will have a larger profile, since more word are included.

Input Format: (Please make sure you use exactly the same input parameter names!)

We use the “[argparse](#)” module to parse the following arguments.

Parameters in **task1_build.py**

1. Input file path (**--train_file**): The path to the train file, including path, file name, and extension.
2. Model file path (**--model_file**): The path to the model file, including path, file name, and extension.
3. Stop word file path (**--stopwords_file**): The path to the provided stopwords file, including path, file name, and extension.
4. Time file path (**--time_file**): The path to the time file, including path, file name, and extension.

Execution example:

```
$ python task1_build.py --train_file <train_file> --model_file <model_file> --stopwords_file  
<stopword_file> --time_out <time_file>
```

```
Example: $ python task1_build.py --train_file train_review_text_150k.json --model_file task1.model  
--stopwords_file stopword --time_out task1_build.time
```

Output Format: task1.model

The model file stores **two things**: a profile for **every business** and a profile for **every user**. Each profile is saved **as one JSON line**. Each line contains either a user ID or a business ID paired with its corresponding list of word profiles.

```
{"user": ["0Q0RS0YptkNM-N9UcDsQNA", ["better", "attentive", "toppings", "key", "brisket", "famous", "credit"]]}  
{"business": ["nUV5XAR_vg-kzhAb4yziEw", ["case", "weeks", "phone", "call", "return", "took", "recommend", "would"]]}
```

Figure 3: An example of task1.model (You can write user and business in the same file)

Step 2: Predict User-Business Preferences

Instructions for task1_predict.py: You will load the model built in Step 1 and predict whether a user prefers to review a business by computing the Jaccard similarity between their profile vectors. Output only valid pairs.

- 1) For each (user, business) pair in val_review.json, compute Jaccard similarity between the user and business word set vectors
- 2) Output only pairs whose **Jaccard similarity is ≥ 0.01**

Input Format:

We use the “[argparse](#)” module to parse the following arguments.

Parameters in **task1_predict.py**

1. Input file path (**--test_file**): The path to the test file, including path, file name, and extension.
2. Model file path (**--model_file**): The path to the model file, including path, file name, and extension.
3. Output file path (**--output_file**): The path to the output file, including path, file name, and extension.
4. Time file path (**--time_file**): The path to the time file, including path, file name, and extension.

Execution example:

```
$ python task1_predict.py --test_file <test_file> --model_file <model_file> --output_file <output_file> --time_file <time_file>
```

Example:

```
$ python task1_predict.py --test_file val_review.json --model_file task1.model --output_file task1.out --time_file task1.build.time
```

Output Format: task1.val.out (**--output_file** in task1_predict.py)

Each line is a JSON record for one valid (user, business) pair:

```
{"user_id": "DN7mB9u36QlCourhzbRq7A", "business_id": "N1ZXhAVz0zbYCBPEXTaUUw", "sim": 0.08362989323843416}  
{"user_id": "WeVp0GZJ-gFux4enlKv4ug", "business_id": "C5Z0zls1hMxRJDjBDV3KoQ", "sim": 0.13250883392226148}
```

Figure 4: An example of task1.val.out

4.2. Grading

We will grade your model based on the number of valid profiles, their correctness, and execution time. For Step 1 (task1_build.py, 2.5pts), we evaluate whether your model contains sufficient profiles (business > 3,000 and user > 55,000), whether your business profiles are correct (overlap ≥ 0.8), and whether your script completes within 45 seconds.

For Step 2 (task1_predict.py, 2.5pts), we evaluate your prediction output task1.val.out by computing precision and recall against the reference output, where

$$\text{Precision} = \# \text{ true positives} / \# \text{ output pairs} \ \&\& \ \text{Recall} = \# \text{ true positives} / \# \text{ ground truth pairs}.$$

Your output must achieve Precision ≥ 0.8 and Recall ≥ 0.7 , your similarity scores must be within 0.05 of the reference on at least 80% of matched pairs, and your script must complete within 30 seconds.

5. Item-based Collaborative Filtering Recommendation System (5pts)

You need to submit the 2 Python scripts on Gradescope (HW4 part2):

- **task2_build.py, task2_predict.py (all in lowercase)**

Please do not change the content in the screenshots (Figure 5 and Figure 6) in your submission beyond adding your code to the main functions. You need to implement the main functions.

[OPTIONAL] You can include other scripts to support your programs (e.g., callable functions).

```
import argparse
import json
import time
import pyspark
import math
import itertools

def main(train_file, model_file, co_rated_thr, sc):

    """
    Your code is here
    """

if __name__ == '__main__':
    start_time = time.time()
    sc_conf = pyspark.SparkConf() \
        .setAppName('hw4_build') \
        .setMaster('local[*]') \
        .set('spark.driver.memory', '4g') \
        .set('spark.executor.memory', '4g')
    sc = pyspark.SparkContext(conf=sc_conf)
    sc.setLogLevel("OFF")

    parser = argparse.ArgumentParser(description='hw4-task2-build')
    parser.add_argument('--train_file', type=str, default='train_review.json')
    parser.add_argument('--model_file', type=str, default='task2.model')
    parser.add_argument('--time_file', type=str, default='task2_build.time')
    parser.add_argument('--m', type=int, default=3)
    args = parser.parse_args()

    main(args.train_file, args.model_file, args.m, sc)
    sc.stop()

    # log time
    with open(args.time_file, 'w') as outfile:
        json.dump({'time': time.time() - start_time}, outfile)
    print('The run time is: ', (time.time() - start_time))
```

Figure 5: task2_build.py template

```

import argparse
import json
import time
import pyspark

def main(train_file, test_file, model_file, output_file, n_weights, sc):
    """
    Your code is here
    """

if __name__ == '__main__':
    start_time = time.time()
    sc_conf = pyspark.SparkConf() \
        .setAppName('hw4_predict') \
        .setMaster('local[*]') \
        .set('spark.driver.memory', '4g') \
        .set('spark.executor.memory', '4g')
    sc = pyspark.SparkContext(conf=sc_conf)
    sc.setLogLevel("OFF")

    parser = argparse.ArgumentParser(description='hw4-task2-predict')
    parser.add_argument('--train_file', type=str, default='train_review.json')
    parser.add_argument('--test_file', type=str, default='val_review.json')
    parser.add_argument('--model_file', type=str, default='task2.model')
    parser.add_argument('--output_file', type=str, default='task2.val.out')
    parser.add_argument('--time_file', type=str, default='task2_predict.time')
    parser.add_argument('-n', type=int, default=3)
    args = parser.parse_args()

    main(args.train_file, args.test_file, args.model_file, args.output_file, args.n, sc)
    sc.stop()

    # log time
    with open(args.time_file, 'w') as outfile:
        json.dump({'time': time.time() - start_time}, outfile)
    print('The run time is: ', (time.time() - start_time))

```

Figure 6: task2_predict.py template

5.1 Task description

In this task, you will build an item-based collaborative filtering (CF) recommendation system using *train_review.json*, which contains two steps:

- Step 1: Compute similarity between business pairs in the review data (task2_build.py)
- Step 2: Predict the ratings for all user-business pairs in a given test file (task2_predict.py)

Step 1: Compute similarity between business pairs

Instructions for build.py: You will write the functions to compute Pearson similarity between business pairs from *train_review.json* and save the required output to **task2.model**.

- Similarity calculation:
 - a. **Inverse User Frequency (IUF) preprocessing:** Before computing similarity between

business pairs, apply IUF weighting to each rating. Businesses that have been rated by many users are less discriminating in determining business similarity - a similar idea to TF-IDF. For each business j , compute: $f_j = \log(n/n_j)$, where n is the total number of users and n_j is the number of users who have rated business j . Transform each rating as: the new rating is defined by the original rating multiplied by f_j before computing similarity. Detailed explanation in 'Recommendation_system_part_2' slide 41.

- b. **Pearson similarity:** Compute Pearson similarity between business pairs using the IUF-weighted ratings. Only consider business pairs that have at least m co-rated users (default: $m=3$). You can find details on Pearson similarity in the slides 'Recommendation_system_part_2', slide 17.
- c. **Co-rated users:** For each pair, keep track of the number of co-rated users they share.

Input Format (Please make sure you use exactly the same input parameter names!)

We use the "[argparse](#)" module to parse the following arguments.

Parameters in **task2_build.py**

1. Input file path (**--train_file**): The path to the train file, including path, file name, and extension.
2. Model file path (**--model_file**): The path to the model file, including path, file name, and extension.
3. Time file path (**--time_file**): The path to the time file, including path, file name, and extension.
4. Threshold (**--m**): The threshold for co-rated users for the business pairs (default: 3)

Execution example:

```
$ python build.py --train_file <train_file> --model_file <model_file> --time_file <time_file> --m  
<co_rated_thr>
```

Example:

```
$ python build.py --train_file train_reviews.json --model_file task2.model --time_file task2_build.time  
--m 3
```

Output Format: task2.model

You must save the generated valid business pairs. For each pair, you will write the business IDs, their Pearson similarity, and their number of co-rated items in the JSON format using exactly the same tags as the example in Figure 7. Each line represents a business pair, i.e., "b1" and "b2". You can truncate decimals for 'sim' if the model file exceeds 100MB, the maximum file size on Gradescope. Due to this size limit on Gradescope, you do not need to include every pair with enough co-rated items and can drop some with low similarity. (e.g., only include the pairs where the similarity ≥ 0.3)

```
{"b1": "cmvseNCdevDdxbx1z3KkWw", "b2": "K7vvEwRd6Y8gRftjeuASuQ", "sim": 0.986, "num_co_rated": 4}  
{"b1": "1WwrdowB8hY-WDA2YWjH1g", "b2": "Hvd8H_ys3ZiKO4DTiH2RJA", "sim": 0.329, "num_co_rated": 23}
```

Figure 7: An example output for the model in the JSON format (task2_build.py)

Step 2: Predict the ratings for all user-business pairs

Instructions for predict.py: You will load the model built in Step 1 and predict ratings for all user-business pairs in the test file.

- **Rating Prediction:** For each user-business pair in the test set, predict the rating using **up to n most similar businesses** to the target business (default: $n = 3$). These n businesses must have been rated by the same user.
- **Downweighting Mechanism:** As proposed by Herlocker et al. you will adjust similarity scores when the number of co-rated users between two businesses is less than 50. Specifically, scale the similarity by a factor of: $\min(1, \text{num_common_users} / 50)$. For example, if two businesses share only 10 users, the similarity is scaled by $10/50 = 0.2$.

Input Format **(Please make sure you use exactly the same input parameter names!)**

Parameters in **predict.py**

1. Input file path (**--train_file**): The path to the train file, including path, file name, and extension.
2. Input file path (**--test_file**): The path to the test file, including path, file name, and extension.
3. Model file path (**--model_file**): The path to the model file, including path, file name, and extension.
4. Output file path (**--output_file**): The path to the output file, including path, file name, and extension.
5. Time file path (**--time_file**): The path to the time file, including path, file name, and extension.
6. Threshold (**--n**): The threshold for co-rated users for the business pairs (default: 3)

Execution example:

```
$ python predict.py --train_file <train_file> --test_file <test_file> --model_file <model_file> --output_file <output_file> --time_file <time_file> --n <threshold>
```

Example:

```
$ python predict.py --train_file train_reviews.json --test_file val_reviews.json --model_file task2.model --output_file task2.val.out --time_file task2.time --n 3
```

Output format

Prediction Format: **task2.val.out** (**--output_file** in task2_predict.py)

You must write the prediction for **EVERY** target pair in the JSON format using exactly the same tags like the example in Figure 4. Each line represents a predicted pair of ("user_id", "business_id"). You can truncate decimals for 'stars' if the result file is larger than 100M.

```
{"user_id": "9fpNC_1mzxCOFOPGu2izCA", "business_id": "-qXHPxXkzTxYSelS4nL0hg", "stars": 5.0}
{"user_id": "c0eR53mlpxkKQXyD_ucRiQ", "business_id": "aITSPNGdEITnvkzCoxHcJQ", "stars": 4.0}
```

Figure 8: An example output for the prediction JSON format

5.2 Grading

We will grade your two models based on the number of valid business pairs in the file and the execution time. To get the full score, your model should contain more than 1,000,000 valid pairs (1pt), the valid pairs should have the roughly correct similarity, num_co_rated (1.5 points each), and the execution time on Gradescope should be less than 400 sec (1pt).

Setting ID	Settings	Number of Pairs	Time
1	co_rated_thr = 3	1,000,000	350s

During the submission period, we will grade your prediction by comparing it against the ground truth in val_review_rating.json. We use RMSE (Root Mean Squared Error) defined in the equation below to evaluate the prediction performance,

$$RMSE = \sqrt{\frac{1}{n} \sum_i w_i (Pred_i - Rate_i)^2}$$

where $Pred_i$ is the prediction for the i -th pair of ("user_id", "business_id") and $Rate_i$ is the true rating for the pair. n is the total number of user and business pairs.

Your script must predict for ALL the pairs in the given test file. We will compute **(1) the overall RMSE**, and **(2) the RMSE for the pairs whose actual ratings are in {1, 2, 5}** because we want to examine the model's capability in predicting good (rating=5) and bad (rating=1,2) reviews.

To get the full score, your results should achieve a lower RMSE and RMSE_125 (2pt each) than the table below, and the execution time on Gradescope should be less than 40 sec.

Settings	RMSE	RMSE_125	Time
n_weights = 3	0.9	1.3	40s

If your RMSE, RMSE_125, and execution time meet the requirements below and the submission is on time (April 20th, 11:59 PM CDT), you can get 1 point [as a bonus during grading \(no partial point\)](#).

Settings	RMSE	RMSE_125	Time
n_weights = 3	0.85	1.1	40s

During the grading period, TAs will grade your code with a test dataset, which has the same data distribution as the val dataset.

6. Grading Criteria

(% penalty = % penalty of possible points you get)

1. You can use your free 5-day extension separately or together. During grading, TAs will count late days **based on the time of submission**. For example, if the due date is 2026/02/24 23:59:59 CDT and your

submission is 2026/02/25 05:23:54 CDT, the number of late days would be 1. **TAs will record grace days by default (NO separate Emails).** However, if you want to save it for next time and treat your assignment as a late submission (with some penalties), **please leave a comment in your submission.**

2. No points will be given if your submission cannot be executed on Gradescope.
3. There is no re-grading. Once the grade is posted on Canvas, we will only re-grade your assignments if there is a grading error. No exceptions.
4. Homework assignments are due at 11:59 pm CT on the due date and should be submitted on Canvas. Late submissions within 24 hours of the due date will receive a 30% penalty. Late submissions after 24 hours of the due date will receive a 50% penalty. Every student has FIVE free late days for homework assignments. You can use these free late days for any reason, separately or together, to avoid the late penalty. There will be no other extensions for any reason. **You cannot use the free late days after the last day of the class.**